Resolução da Lista de Exercícios

A Efemeridade do In-Memory"

Os dados ainda estão lá?

Não. Após rodar o *dotnet-run* novamente a lista volta vazia.

Por que acontece?

Porque o banco In-Memory é armazenado diretamente na memória RAM do computador, quando é feita uma nova requisição esses dados são apagados e reconstruídos.

Semântica HTTP

Tente realizar um POST enviando um objeto vazio. O que acontece?

Retorna erro 400 Bad Request. O sistema tenta ler o JSON, mas como o documento está vazio e não seguindo as necessidades de preenchimento dos campos ele retorna que nada foi achado.

Tente realizar um DELETE em um ID que não existe (ex: ID 99). Sua API retornou 204 No Content ou 404 Not Found? Qual seria o comportamento ideal para um sistema distribuído robusto?

O DELETE retornará um 404, de que não achou nada com aquela numeração. Mas dentro de um sistema robusto ele deveria retornar error 204, dizendo que a requisição foi processada com sucesso, mas não ha nada para ela retornar.

Idempotência na Prática:

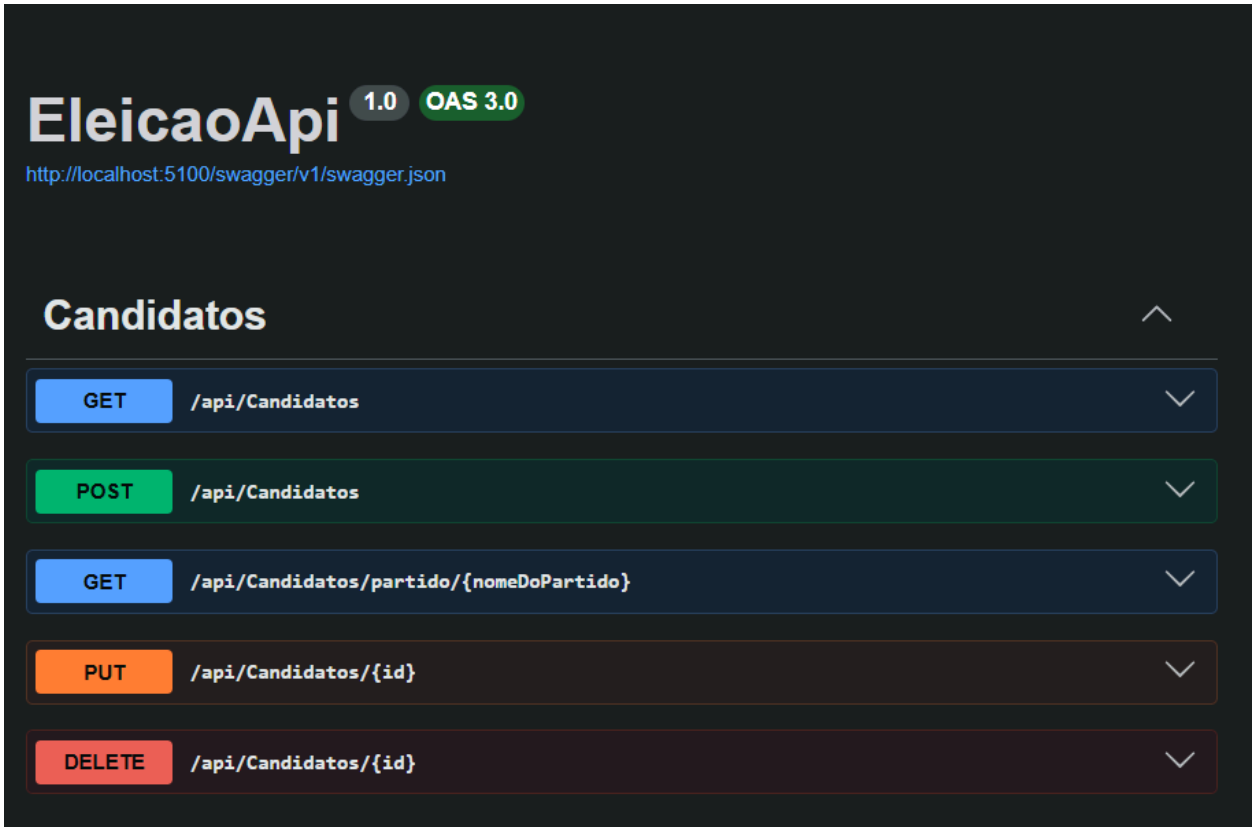
Execute o método DELETE para o ID 1 duas vezes seguidas. O resultado da segunda execução foi diferente da primeira no que diz respeito ao estado do banco de dados? Isso prova que o DELETE é um método idempotencia? Justifique.

Não. Após a primeira execução, o registro foi removido. A segunda execução tentou remover algo que já não existia, mantendo o estado do banco exatamente igual: "registro deletado".

Sim, uma operação é idempotente se após ser atualizado no servidor seu valor sempre retorna o mesmo independente de quantas vezes a requisição seja feita.

Embora o código de status HTTP possa mudar (a primeira vez retorna 204 e a segunda 404), o estado do recurso no servidor não muda após a primeira chamada bem-sucedida. O registro continua inexistente.

REQUISIÇÕES



POST

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5100/api/Candidatos' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 0,
  "nome": "Jose Carlos",
  "partido": "TP",
  "numero": 5555,
  "viceNome": "Joaquim da Logoa"
}'
```

Request URL

```
http://localhost:5100/api/Candidatos
```

Server response

Code	Details
200	Response body

```
{
  "id": 1,
  "nome": "Jose Carlos",
  "partido": "TP",
  "numero": 5555,
  "viceNome": "Joaquim da Logoa"
}
```

Download

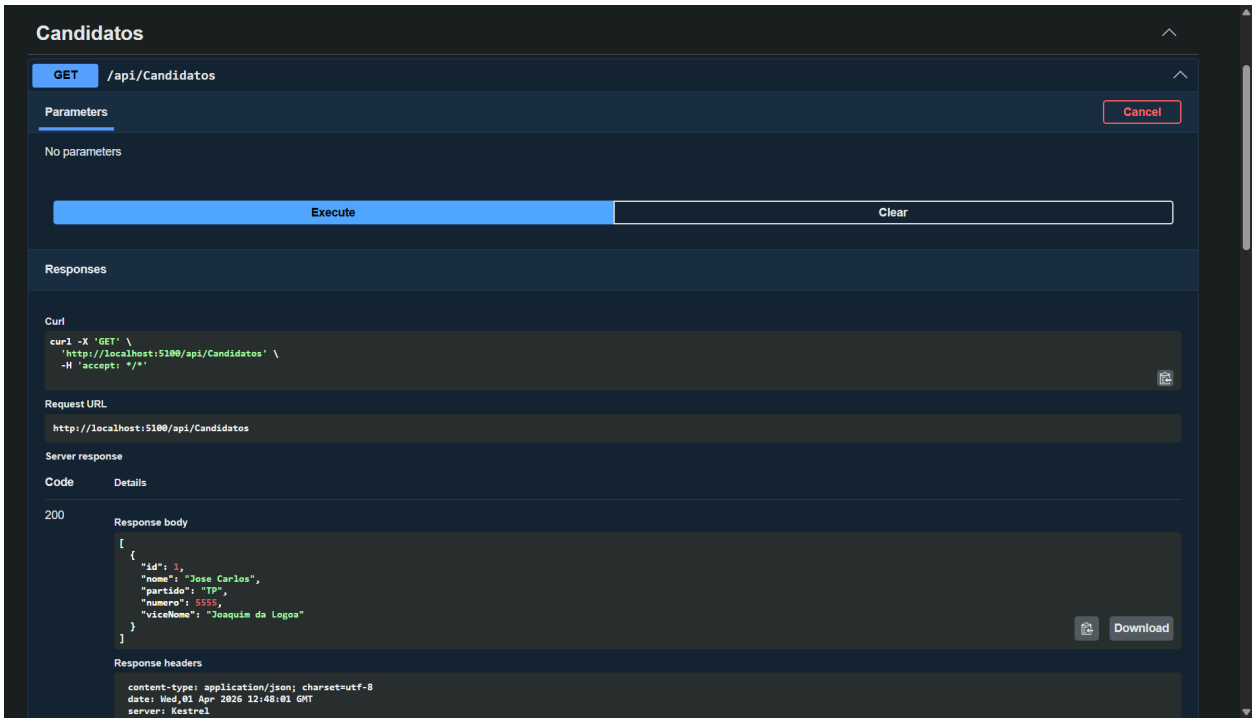
Response headers

```
content-type: application/json; charset=utf-8
date: Wed,01 Apr 2026 12:47:21 GMT
server: Kestrel
transfer-encoding: chunked
```

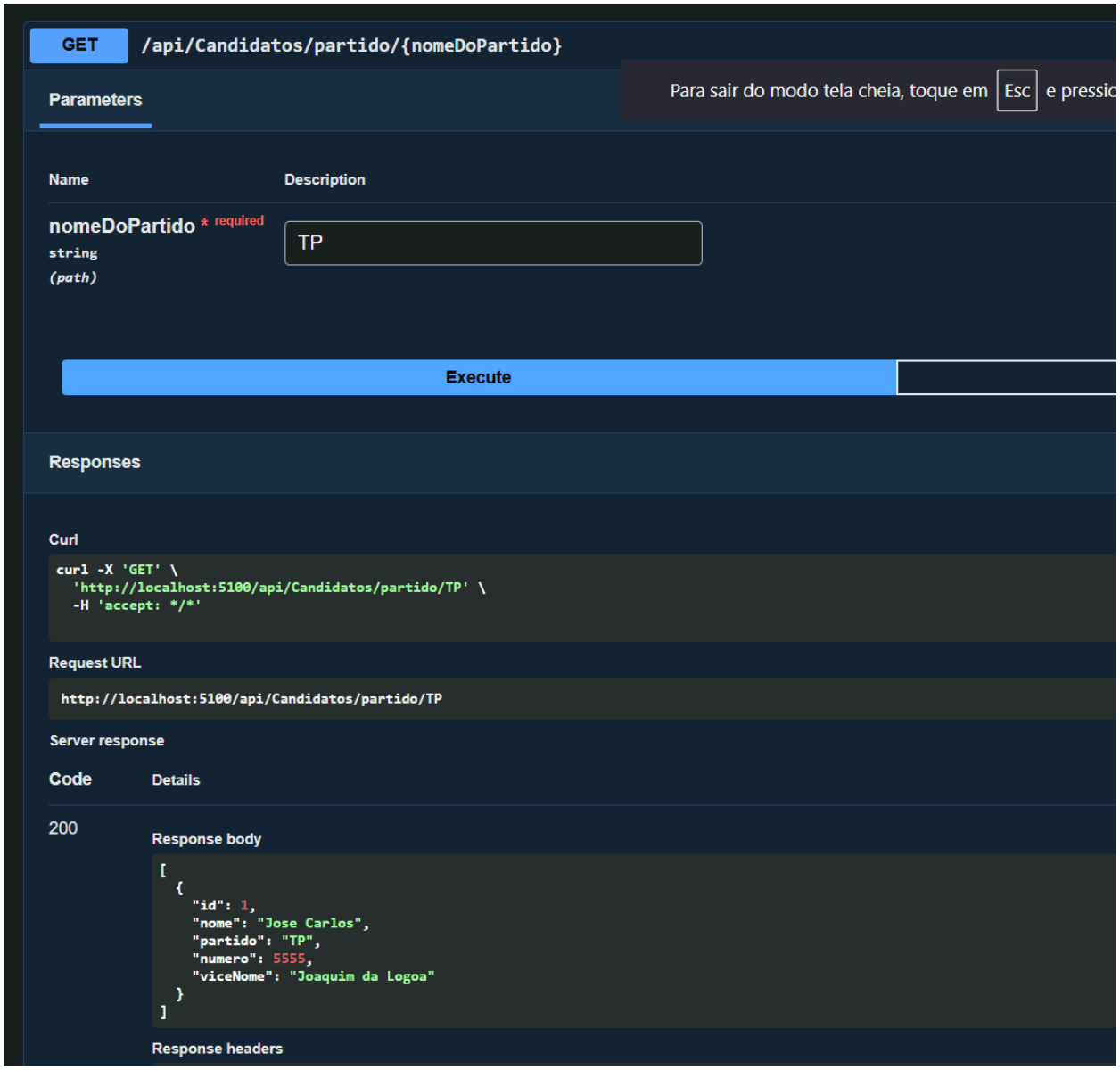
Responses

Code	Description	Links
200	OK	*No links*

GET ALL


Candidatos
GET
/api/Candidatos
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:5100/api/Candidatos' \
-H 'accept: */*'
Request URL
http://localhost:5100/api/Candidatos
Server response
Code
Details
200
Response body
[
{
"id": 1,
"nome": "Jose Carlos",
"partido": "TP",
"numero": 5555,
"viceNome": "Joaquim da Logoa"
}
]
Download
Response headers
content-type: application/json; charset=utf-8
date: Wed,01 Apr 2026 12:48:01 GMT
server: Kestrel

GET BY NOME DO PARTIDO



PUT

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:5100/api/Candidatos/2' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 2,
  "nome": "Tiao da Obra",
  "partido": "TC",
  "numero": 4444,
  "viceNome": "Joaquim do Mar Afogado"
}'
```

Request URL

```
http://localhost:5100/api/Candidatos/2
```

Server response

Code	Details
204 *Undocumented*	Response headers

```
date: Wed,01 Apr 2026 12:51:34 GMT
server: Kestrel
```

Responses

Code	Description
200	OK

DELETE

DELETE /api/Candidatos/{id}

Parameters

Name	Description
id * required integer($int32) *(path)*	2

Execute

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:5100/api/Candidatos/2' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:5100/api/Candidatos/2
```

Server response

Code	Details
204 *Undocumented*	Response headers date: Wed,01 Apr 2026 12:52:31 GMT server: Kestrel

Responses

Code	Description
200	OK